examto

Application Security Learning Lab

The Complete Step-by-Step Guide

Everything you need to finish every room on your own. Free, runs entirely in GitHub Codespaces, no interaction with anyone required.

- 23 hands-on rooms across 4 tracks (foundations, web exploits, AppSec, DevSecOps)
- 6 sections per room: Brief, Theory, Worked example, Knowledge check, Investigation, Capture
- Live terminal on the right - the Codespace is your machine
- Real free tools: Bandit, Semgrep, Checkov, pip-audit, sqlmap, gitleaks
- A separate learn-python track that tests you lesson by lesson

Built for your path: from infrastructure/cloud/endpoint security into remote Application Security and DevSecOps roles. Keep this PDF handy; it is your map.

0. What you are holding

examto is a free, self-service security lab that lives in your GitHub repository. It looks and works like TryHackMe / Hackversity: theory on the left, a lab and a live terminal on the right. You read, you try, you capture a flag or answer, you move on. This guide takes you from a cold start to finishing all 23 rooms without needing to ask anyone anything.

> **The one rule to remember**
> You never pay for anything and you never wait for anyone. If a step asks you to buy a VM or sign up for a paid tier, you are off the path - come back here.

1. One-time setup (about 3 minutes)

You only do this once. It runs in your browser - nothing is installed on your own computer.

1. Open your repo: github.com/kingkish269-ship-it/examto
2. Click the green 'Code' button, choose the 'Codespaces' tab, then 'Create codespace on main'.
3. Wait for it to build. The .devcontainer in the repo auto-installs Python and the lab dependencies the first time.
4. When the editor loads, you have a full Linux machine in your browser. This Codespace IS the lab machine you will attack and scan.

> **Free hours**
> GitHub gives personal accounts a monthly pool of free Codespaces hours. It is plenty for this course if you STOP the Codespace when you finish each session (see section 9). You will not be charged on the free tier.

2. Start the web lab

Every time you come back, open the Codespace again (github.com -> your repo -> Code -> Codespaces -> click the existing one). Then in the terminal at the bottom of the editor, pull the latest content and start the lab with a single command:

```
git pull
bash webapp/start.sh
```

start.sh installs the small web dependencies (flask, flask-sock) if they are missing and then launches the server. You will see a message that it is running on port 5000.

1. A 'port forwarded' popup appears in the bottom-right. Click 'Open in Browser'. If you miss it, open the 'Ports' tab and click the globe icon next to port 5000.
2. The lab opens in a new browser tab: room list on the left, room content in the middle, and a live terminal on the right.
3. The terminal connects automatically. You do not need to press Ctrl+` or do anything in VS Code - the terminal in the page is the one you use.

> **If the terminal says '[terminal closed]'**
> That only happens when flask-sock did not install. Stop the server (Ctrl+C in the editor terminal) and re-run 'bash webapp/start.sh' - it installs the backend and the terminal then connects on its own.

3. How a room works (the 6 sections)

Every room has the same six sections. Work them top to bottom. The left pane shows the content; use the terminal on the right to actually do the work.

Section	What you do
1. Brief	A two-line 'why this matters'. Sets the scene.
2. Theory	Four parts: What it is, How it works, In the real world (real breaches), and How to prevent it. This is the part you study.
3. Worked example	A fully worked command/response so you see exactly what 'good' looks like before you try.
4. Knowledge check	Multiple-choice / short answers. You must get them right to mark the section done. Instant grading.
5. Investigation	A complete hands-on lab in the terminal: exploit the flaw, apply the fix, then re-test to confirm it is closed. Hints provided.
6. Capture	Enter the flag or the one-word answer you derived. Graded against a hash - the answer is never stored in plaintext.

A room counts as complete when all six sections are marked done. Your progress is saved locally in the Codespace so you can resume.

> **Finish the whole room, not one command**
> The Investigation section lists several steps for a reason. Run all of them. The Capture answer usually only makes sense after you have actually done the investigation, not after a single curl.

4. The full room order

Do them in this order. Each track builds on the one before it. Rooms 1-2 give you the mindset, 3-10 are the classic web exploits you must be able to demonstrate, 11-14 are application-security practice, and 15-23 are the DevSecOps tooling that gets you hired for pipeline and platform security roles.

#	Room	Track	Core idea
1	Intro to AppSec	Foundations	Source-and-sink mental model; the golden rule
2	Broken Access Control	Foundations	The #1 OWASP risk; IDOR; server-side checks
3	SQL Injection	Web Exploits	UNION/boolean injection; parameterized queries
4	Command Injection	Web Exploits	Shell metacharacters; avoid shell=True
5	Cross-Site Scripting	Web Exploits	Reflected/stored XSS; output encoding; CSP
6	SSRF	Web Exploits	Server-side request forgery; cloud metadata
7	Broken Authentication	Web Exploits	Sessions, JWT alg=none, token forging
8	Crypto Failures	Web Exploits	Weak hashing, secrets, transport security
9	Security Misconfiguration	Web Exploits	Headers, defaults, verbose errors
10	Insecure Deserialization	Web Exploits	pickle/yaml RCE; safe parsing
11	API Security	AppSec	BOLA/BFLA; OWASP API Top 10
12	AI / LLM Security	AppSec	Prompt injection; OWASP LLM Top 10
13	Threat Modeling	AppSec	STRIDE; data-flow diagrams

14	Secure Code Review	AppSec	Spotting vulnerable patterns by hand
15	SAST	DevSecOps	Static analysis with Bandit / Semgrep
16	DAST	DevSecOps	Dynamic scanning of a running app
17	Dependency Scanning (SCA)	DevSecOps	pip-audit; known-CVE components
18	Secret Scanning	DevSecOps	gitleaks; committed-credential detection
19	CI/CD Pipeline Security	DevSecOps	Build-time gates; failing on findings
20	IaC Security	DevSecOps	Checkov on Terraform; misconfig rules
21	Container Security	DevSecOps	Non-root, image hygiene
22	Kubernetes Security	DevSecOps	securityContext; privileged pods
23	Supply Chain Security	DevSecOps	SBOM; provenance; dependency risk

5. The Python track (do this in parallel)

You told me Python is your weak spot. There is a separate, self-contained track that teaches exactly the Python an AppSec engineer needs, and tests you after each lesson. It does not need the web lab - run it straight in the Codespace terminal.

```
cd /workspaces/examto/learn-python
cat README.md          # the plan and lesson list
python3 check.py       # runs the tests for each lesson you have done
```

- 14 lessons from variables up to calling HTTP APIs, including a lesson on the Microsoft Graph / Intune style API calls that match your background.
- Each lesson has an exercise file you edit; check.py grades it and tells you what is still failing.
- Aim for one lesson per day alongside one or two rooms. By the time you reach the SAST/review rooms the Python will feel natural.

6. Using the real free tools

The rooms teach the logic with small offline stand-ins so they always run with zero setup. When you want to run the REAL industry tools against the lab code (the exact tools you will use on the job), use the tools-lab.

```
cd /workspaces/examto/tools-lab
cat README.md          # what each tool is and when to use it
bash install.sh        # installs the real tools into a local venv
bash run_real_tools.sh # runs them all against the lab app and prints findings
```

Tool	Type	What it catches in this repo
Bandit	SAST (Python)	pickle, subprocess shell=True, md5, raw SQL, urlopen
Semgrep	SAST (rules)	Multiple injection and crypto patterns
pip-audit	SCA	Known CVEs in the pinned dependencies
Checkov	IaC	Terraform misconfigurations (public DB, open SG, etc.)
sqlmap	DAST	Confirms the SQL injection in the lab app

Tool		
gitleaks	Secrets	Committed credentials / tokens

> **Note on Semgrep in a fresh Codespace**
> Semgrep needs a writable home for its settings. If it complains, run 'export SEMGREP_SETTINGS_FILE=$PWD/.semgrep.yml' first. The run_real_tools.sh script already handles this for you.

7. A suggested weekly rhythm

You do not have to rush. A sustainable pace that finishes everything in about 4-5 weeks:

Week	Rooms	Python	Tools
1	1-2 (foundations) + 3-5	Lessons 1-4	Bandit intro
2	6-10 (rest of web exploits)	Lessons 5-8	sqlmap, Semgrep
3	11-14 (AppSec)	Lessons 9-11	Code review with Bandit
4	15-19 (SAST/DAST/SCA/secrets/CICD)	Lessons 12-14	pip-audit, gitleaks
5	20-23 (IaC, containers, k8s, supply chain)	Review	Checkov

8. Prove it (build your evidence as you go)

Hiring managers for 70+ LPA remote AppSec/DevSecOps roles want proof, not a certificate list. As you finish rooms, capture evidence:

- Keep a short notes file per room: the vuln, how you exploited it, the fix. This becomes your talking points in interviews.
- Screenshot real tool output (Bandit/Checkov/sqlmap findings) - that is portfolio material.
- Push a 'writeups' folder to this same repo. A public repo of security writeups is a strong signal.
- Map each room to the job description language: 'Broken Access Control', 'SAST in CI', 'IaC scanning', 'SBOM' are phrases recruiters search for.

9. Stop the Codespace when you are done (saves your free hours)

This is the one piece of housekeeping that keeps everything free. Codespaces bill by the hour they are RUNNING, so stop it at the end of each session.

1. In the Codespace editor, press F1 (or Ctrl+Shift+P) to open the command palette.
2. Type 'Codespaces: Stop Current Codespace' and press Enter.
3. Or, from github.com/codespaces, click the '...' next to your codespace and choose 'Stop codespace'.

> **Stopping is not deleting**
> Stopping keeps all your files and progress; it just powers the machine down so the clock stops. Next session you re-open it and pick up where you left off. Only use 'Delete' when you are completely finished with the course.

10. Quick command reference

The handful of commands you will actually use, in one place:

```
# start / resume the web lab
git pull && bash webapp/start.sh      # then open port 5000

# python track
cd learn-python && python3 check.py

# real security tools against the lab
cd tools-lab && bash install.sh && bash run_real_tools.sh

# stop a stuck server
Ctrl+C in the editor terminal

# stop the codespace (saves hours):  F1 -> Codespaces: Stop
```

11. Job-readiness checklist

You are ready to apply confidently when you can tick all of these:

- [ ] Finished all 23 rooms (every section, every capture).
- [ ] Finished the learn-python track - check.py passes all lessons.
- [ ] Ran every real tool in tools-lab and can explain each finding.
- [ ] Can explain Broken Access Control, SQLi, XSS, SSRF and the fix for each from memory.
- [ ] Can describe a DevSecOps pipeline: SAST + SCA + secret scan + IaC scan with a gate that fails the build.
- [ ] Have a public writeups repo and 3-4 screenshots of real tool output.
- [ ] Can tie your existing SC-100 / AWS Security / Zscaler ZTCA background to application and pipeline security in one sentence.

> **Your edge**
> Most AppSec candidates come from dev and are weak on cloud/network/endpoint. You are the opposite, and you now have the application + pipeline skills to match. That combination is exactly what platform-security and DevSecOps teams struggle to hire. Finish the rooms, show the evidence, and lean on it.